

LegacyLift

An Agentic Legacy Modernization Accelerator with Test-Backed Safety,
Powered by IBM Bob IDE

IBM Bob 2.0 Hackathon (lablab.ai) | 25-27 September 2026 | Team: [Team Name] | Members: [Name 1], [Name 2], [Name 3]

Abstract

Modernizing legacy software is slow, expensive and risky. Engineers must first understand undocumented code, then change it without breaking behaviour that nobody wrote down. LegacyLift is an agentic workflow built on IBM Bob IDE that turns this multi-week manual effort into a guided, test-backed pipeline. Given a legacy Java repository, LegacyLift (1) explains its architecture and dependencies, (2) generates characterization tests that lock in current behaviour, (3) migrates the code module by module to a modern stack (e.g. Java 8 to Java 21, javax to jakarta, legacy Spring to Spring Boot 3), and (4) validates the result by re-running the tests and producing a risk and migration report. The central idea is *tests as proof*: every migrated module ships with evidence that behaviour was preserved. We measure impact through time saved, test coverage before and after migration, test pass rate and the number of issues caught before release.

1. Introduction

1.1 Problem. Many organizations still run business-critical systems on outdated frameworks and language versions. These systems are hard to change because (a) the original authors have left, (b) documentation is missing or stale, and (c) automated tests are thin or absent. As a result, modernization projects are frequently delayed, over budget, or abandoned. The developer workflow that suffers most is *maintenance and migration*: time, effort and error rates are all high today.

1.2 Why now. AI coding assistants can already autocomplete lines of code, but migration needs more than autocomplete. It needs an agent that can read an entire repository, plan multi-step changes, generate and run tests, and explain its reasoning. IBM Bob IDE provides agent mode, parallel tasks, subagents and document understanding, which match these needs.

1.3 Objectives and contributions.

- **Explain:** automatic architecture map, dependency graph and plain-English module summaries for an unfamiliar legacy repository.
- **Protect:** characterization tests generated *before* any change, so behaviour is locked in.
- **Migrate:** module-by-module modernization driven by Bob agent mode, with rule-based tools used for mechanical changes.
- **Prove:** validation by re-running tests, plus a before/after report with coverage, pass rate and risk summary.
- **Measure:** a reproducible evaluation that compares manual effort against the LegacyLift workflow.

2. Related Work

Note to team: this section has reserved space. The short entries below are a starting point; expand each with proper citations and a clear statement of how LegacyLift differs.

2.1 Rule-based code migration

Tools such as OpenRewrite apply deterministic refactoring recipes (for example, javax to jakarta or Spring Boot upgrades) across a codebase. They are reliable for mechanical changes but cannot explain code, reason about business logic, or handle cases that no recipe covers. **Gap:** no understanding or behaviour verification.

[Space reserved: add 2-3 more rule-based or compiler-based migration approaches and compare.]

2.2 AI-assisted code modernization

Industry assistants such as IBM watsonx Code Assistant for Z (COBOL to Java), Amazon Q Developer code transformation (Java version upgrades) and GitHub Copilot provide AI support for modernization. Most focus on code generation or a single transformation type. **Gap:** limited emphasis on generating behaviour-locking tests before migration and reporting evidence afterwards.

[Space reserved: add academic work on LLM-based code translation and migration, with results and limitations.]

2.3 Automated test generation

Search-based tools such as EvoSuite and random testing tools such as Randoop generate unit tests for Java automatically. More recent work uses large language models to generate readable tests. **Gap:** these tools are not usually integrated into an end-to-end migration workflow.

[Space reserved: add LLM-based test generation studies and coverage results.]

2.4 Characterization testing and agentic software engineering

Feathers (2004) introduced *characterization tests*: tests that record what legacy code currently does, so it can be changed safely. Benchmarks such as SWE-bench evaluate LLM agents on real repository-level tasks. LegacyLift combines both ideas: an agent that writes characterization tests first and then performs the migration.

[Space reserved: add agent frameworks, multi-agent coding systems and relevant benchmarks.]

2.5 Summary of positioning

Approach	Explains code	Generates tests	Migrates	Proves behaviour
Rule-based (OpenRewrite)	No	No	Yes	No
AI assistants (general)	Partly	Partly	Yes	Rarely
Test generators (EvoSuite, Randoop)	No	Yes	No	Partly
LegacyLift (ours)	Yes	Yes	Yes	Yes

[Space reserved: team may add more rows or a paragraph on novelty.]

3. Proposed System

3.1 Workflow overview

LegacyLift runs as a four-stage pipeline. Each stage is a separate Bob task, so stages can be split between team members and run in parallel where possible.

Stage	Input	What happens	Output
1. Explain	Legacy repo	Bob agent reads the repo, maps modules, dependencies and entry points, and writes plain-English summaries.	ARCHITECTURE.md, dependency graph (Mermaid), module summaries
2. Protect	Repo + Stage 1 map	Bob generates characterization tests (JUnit 5) for critical modules and records current outputs. JaCoCo measures baseline coverage.	Test suite, baseline coverage report
3. Migrate	Repo + tests	OpenRewrite handles mechanical changes; Bob agent handles the rest module by module (APIs, config, deprecated libraries).	Migrated code on a separate branch
4. Prove	Migrated code + tests	Tests re-run on the new stack; failures are explained and fixed by Bob; final risk report is generated.	MIGRATION_REPORT.md, dashboard metrics

3.2 Architecture

The system has three layers:

- **Agent layer (IBM Bob IDE):** agent mode performs repository analysis, test generation and migration. Subagents handle independent modules in parallel. Custom rules and prompts keep output consistent across tasks.
- **Tooling layer:** JDK 8 and JDK 21, Maven or Gradle, JUnit 5, Mockito, JaCoCo and OpenRewrite. These do deterministic work so Bobcoins are spent only on reasoning.
- **Reporting layer:** Python scripts collect metrics; a Streamlit dashboard shows before/after results, architecture diagrams and the risk report for the demo.

[Space reserved: insert architecture diagram here (export from Mermaid or draw.io).]

4. Role of IBM Bob IDE

IBM Bob IDE is the core of LegacyLift, not an add-on. Every stage that needs understanding or judgement runs through Bob:

- **Agent mode** for multi-step, repository-wide tasks (analysis, migration, fixing failing tests).
- **Parallel tasks and subagents** to process independent modules at the same time, reducing total runtime.
- **Document understanding** to read existing README files, configuration and framework migration guides, and apply them to the code.

- **Evidence:** every Bob task used for the submission is screenshotted (task header, session consumption summary) and stored in the `bob_sessions/` folder, named e.g. `team_task01_repo_analysis_summary.png`.

5. Tools and Technology Stack

Category	Tool	Purpose
Core (required)	IBM Bob IDE v2.0.2+	Main AI agent: analysis, tests, migration, fixes
Core (optional)	Bob Shell	Scripting and automating Bob tasks from the command line
Language / build	JDK 8, JDK 21, Maven or Gradle	Run legacy code, build and run migrated code
Migration	OpenRewrite	Deterministic recipes for mechanical upgrades (saves Bobcoins)
Testing	JUnit 5, Mockito	Characterization and unit tests
Coverage	JaCoCo	Baseline and post-migration coverage metrics
Analysis	Python scripts	Collect metrics, build dependency data
Diagrams	Mermaid / Graphviz	Auto-generated architecture and dependency diagrams
Dashboard	Streamlit (or React)	Before/after results for the demo
Backup AI	IBM watsonx.ai (optional)	Summaries and explanations if Bobcoins run out
Collaboration	Git + GitHub	Code repository, includes <code>bob_sessions/</code> folder
Demo	OBS Studio / Loom	Record the demo video

6. Datasets and Compliance

We use only public open-source repositories whose licences allow commercial use. No client data, company confidential data, personal information or social media content is used. Every source is listed in `DATA_SOURCES.md`.

Source (candidate)	Licence	Use
Older Spring PetClinic version (GitHub)	Apache-2.0	Primary legacy Java target
Java EE / older Spring sample apps (GitHub)	Apache-2.0 / MIT (verify)	Secondary test targets
Open Mainframe Project COBOL samples (optional)	Verify per repo	Stretch goal: COBOL explanation
Official framework migration guides	Public documentation	Input for Bob document understanding

Action: confirm the exact licence of each repository before use and record the URL, licence and access date.

7. Implementation Plan (48 Hours)

Time	Milestone	Owner
0-6 h	Pick target repo, confirm licences, set up Bob (hackathon account), repo skeleton, <code>bob_sessions/</code> folder	All
6-16 h	Stage 1 Explain: architecture map, dependency graph, module summaries	Member 1

Time	Milestone	Owner
6-24 h	Stage 2 Protect: characterization tests, baseline coverage	Member 2
24-38 h	Stage 3 Migrate: OpenRewrite + Bob agent migration, module by module	Member 3
30-40 h	Stage 4 Prove: re-run tests, fix failures, migration report	Members 2 and 3
38-44 h	Dashboard, metrics collection, diagrams	Member 1
44-48 h	Demo video, README, final bob_sessions check, submission	All

7.1 Bobcoin budget (40 per member)

Member	Main Bob tasks	Planned coins	Reserve
Member 1	Repo analysis, architecture explanation, report text	~30	~10
Member 2	Test generation, failing test analysis	~30	~10
Member 3	Migration, fixing compile and test errors	~32	~8

Rules: write one clear, detailed prompt per task instead of many short messages; use OpenRewrite and scripts for mechanical work; check usage in Settings, General; switch to watsonx.ai for non-core text if coins run low.

8. Evaluation and Success Metrics

We compare a manual baseline (estimated from team effort on one module) with the LegacyLift workflow on the same repository. Target values are goals, to be replaced with measured results.

Metric	How measured	Target	Result
Time to understand repo	Manual reading vs Stage 1 output	Hours to minutes	[TBD]
Test coverage before migration	JaCoCo, before vs after Stage 2	+30 points or more	[TBD]
Test pass rate after migration	Stage 4 test run	95% or more	[TBD]
Modules migrated	Count on migrated branch	All core modules	[TBD]
Total migration time	Manual estimate vs pipeline run	70% reduction or more	[TBD]
Issues caught before release	Failing tests found and fixed	Reported	[TBD]
Bobcoins used	Admin dashboard	Within team budget	[TBD]

9. Expected Impact, Risks and Limitations

Impact. Developers get a trustworthy path from unknown legacy code to a modern stack, with documentation, tests and a report as by-products. Teams spend less manual effort, make fewer regressions and can show reviewers evidence that behaviour is preserved.

Risk / limitation	Mitigation
Bobcoins run out	Split tasks across members, use scripts and OpenRewrite for mechanical work, watsonx.ai as backup
Generated tests may miss edge cases	Focus on critical modules; report coverage honestly; list untested areas
Target repo too large for 48 h	Scope to core modules; treat others as future work

Risk / limitation	Mitigation
Legacy code fails to build	Pick a repo that builds on JDK 8 during hour 0-6
Only Java is supported	COBOL explanation as a stretch goal; other languages as future work

10. Conclusion and Future Work

LegacyLift shows how IBM Bob IDE can move beyond autocomplete into a full modernization workflow: explain, protect, migrate and prove. Future work includes COBOL and other legacy languages, CI integration so every migration opens a pull request with its report, and a larger study on real-world repositories.

11. Submission Checklist

- Public GitHub repository with source code, README and setup steps
- bob_sessions/ folder with a PNG summary screenshot for every Bob task
- DATA_SOURCES.md listing every dataset or repository with licence
- Demo video showing all four stages and the dashboard
- This proposal with measured results filled in

References

- [1] M. Feathers, *Working Effectively with Legacy Code*. Prentice Hall, 2004.
- [2] OpenRewrite, automated source code refactoring. <https://docs.openrewrite.org>
- [3] G. Fraser and A. Arcuri, "EvoSuite: automatic test suite generation for object-oriented software," in *Proc. ESEC/FSE*, 2011.
- [4] C. Pacheco and M. D. Ernst, "Randoop: feedback-directed random testing for Java," in *OOPSLA Companion*, 2007.
- [5] C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.
- [6] IBM, IBM Bob IDE Hackathon Guide, 2026.
- [7] [Team: add further references for Related Work here.]